

Performance Evaluation of Apache Hadoop, Spark, and Flink for Batch Processing of Big Data: A Comparative Analysis

Lakshmana Kumar Yenduri

*Senior Staff Software Engineer, Payment Products Development at Visa Inc| IEEE Senior Member
San Francisco Bay Area, California, 94404, USA*

lakshmanyenduri@gmail.com

Abstract—The work explores the demand for Big Data processing and delves into the functioning of large-scale data processing architectures, focusing on batch and real-time processing. The experiment conducted analyzes the execution time of Apache Hadoop (AH), Apache Spark (AS), and Apache Flink (AF) tools. Results indicate that Spark outperformed Flink and Hadoop across all experiments, demonstrating notable speed advantages. In the first experiment with a 1 GB data source, Spark was 186% faster than Flink and 251% faster than Hadoop. Similarly, in the second experiment with a 3 GB data source, Spark surpassed both competitors, being 233% faster than Hadoop and 334% faster than Flink. Processing a 5 GB data source further highlighted Spark's superiority, with a 197% improvement over Hadoop and 316% over Flink. Despite the absence of parallelism and a distributed execution environment, the findings unanimously conclude that Spark achieved superior performance in batch processing large datasets in a pseudo-distributed cluster setting.

Keywords— Big Data, Speed, Batch processing, Performance evaluation.

I. INTRODUCTION

In the era of Big Data, efficient data processing architectures are crucial for timely analysis of vast datasets to extract valuable insights. Apache Hadoop (AH), Apache Spark (AS), and Apache Flink (AF) are prominent contenders in large-scale data processing [1-2]. This study focuses on batch processing to evaluate their performance, aiming to shed light on their execution time with large datasets. Through experiments ranging from 1 GB to 5 GB, AS emerged as the frontrunner, showing significant performance advantages over AF and AH. Despite the absence of parallelism, Spark maintained its lead, indicating its potential for scalable batch processing. Further research is needed to explore Spark's performance in distributed environments. Overall, this study underscores Spark's significance in batch processing large datasets, contributing to our understanding of Big Data processing and informing data analytics workflows [3-4].

The evolution of technology has led to an unprecedented surge in data generation across various sectors, presenting both opportunities and challenges for organizations [5]. In response, researchers and practitioners have delved into the realm of big

data analytics to extract valuable insights and drive informed decision-making. This introduction provides an overview of recent advancements in big data analytics, as evidenced by a selection of scholarly articles [6-7].

In today's data-driven world, the integration and analysis of vast amounts of data have become crucial for advancing various sectors, from public administration to financial security and industrial manufacturing. The challenges of implementing effective data integration strategies in the public sector highlight the need for robust frameworks and methodologies. Comprehensive overviews of big data analytics reveal the latest advancements, providing critical insights into the architecture and technologies that underpin these systems [8].

Cybersecurity remains a paramount concern, particularly with the rise of IoT-based systems, which are vulnerable to sophisticated threats such as distributed denial of service (DDoS) attacks. Innovative algorithms and methodologies are being developed to detect and mitigate these threats, enhancing security across various domains, including banking and network infrastructure. In parallel, the financial sector benefits from advanced analytical techniques to combat credit card fraud, leveraging powerful tools like AS [9].

Optimizing data processing workflows is another critical area of focus, with research exploring efficient data partitioning and sampling methods. Additionally, multimodal data analytics integrates diverse signals to provide a more comprehensive understanding of complex phenomena, extending the capabilities of traditional data analysis.

In the industrial sector, the adoption of big data and stream processing platforms is essential for the evolution of Industry 4.0, particularly in predictive maintenance and other advanced applications. The development of efficient feature reduction methods for detecting security threats further underscores the ongoing efforts to enhance network security and reliability [10].

Overall, the diverse applications and innovative methodologies within big data analytics highlight its transformative impact, addressing contemporary challenges and driving progress across various fields.

II. LITERATURE REVIEW

In study [11], recent advancements in academic literature on Operations and Risk Management (ORM) from a data analytics perspective were explored. Findings were summarized, and a framework for implementing data-driven ORM was proposed, outlining future research directions.

In article [12], an exploration was made into how big data analytics can address significant strategic human capital questions, enhancing overall firm performance within the Human Resources (HR) domain. The discussion revolved around how new data sources enabling real-time assessment of workforce performance can aid in identifying and developing knowledge stars that contribute disproportionately to firm performance. Regulatory and ethical challenges needing addressing for successful big data analytics implementation in HR. The article concluded by discussing how big data analytics can facilitate strategic changes within HR and the organization as a whole.

This paper [13] delved into the origins of machine learning algorithms, a topic rarely discussed but of paramount importance to enthusiasts and amateurs alike. The LSTM network and SVM algorithm were thoroughly reviewed and analyzed, showcasing their superior behavior compared to other algorithms. Various aspects were addressed qualitatively and quantitatively, ranging from origin and definition to execution methodologies and real-time applications. An extensive comparison of algorithm performances on quantitative and qualitative grounds was also presented.

Similarly, in paper [14] the twelve typical platforms were introduced, and their implementation principles were analyzed. Big data analysis algorithms were categorized according to MapReduce implementation. Factors affecting algorithm performance were identified, and optimization methods for external memory algorithms were proposed, offering valuable insights for researchers in the field.

Lastly, in study [15], a comprehensive introduction was provided to streaming feature selection techniques in machine learning. Traditional feature-selection algorithms were reviewed, and current algorithms using streaming feature selection were scrutinized, highlighting their strengths, weaknesses, and ongoing challenges in big data research.

III. METHOD

The method employed in this study aligns with Garcia's definition of a systematic approach involving reflection and experimentation to achieve predefined research objectives. This chapter outlines the methodological procedures used to fulfill the objectives of the study, including the research type, solution scheme, and work limitations.

The research adopts a qualitative and applied approach with comparative analyses and statistical calculations to evaluate Big Data tools' practical use. Qualitative research allows for in-depth exploration and produces illustrative information, vital for understanding complex phenomena.

The research follows an exploratory path, aiming to enhance familiarity with the problem and construct hypotheses. It combines bibliographical research and case studies to achieve its objectives.

The methodological steps include:

- Problem Definition: Identifying the problem related to Big Data processing.
- Theoretical Foundation: Presenting the theoretical fundamentals of Big Data and related solutions.
- Solution Proposal: Developing a solution to the problem, describing the experiment's steps and tools.
- Implementation: Executing the proposed solution and providing a detailed guide.

The objective is to explore Big Data tools' utilization by developing a word counter program using AH, Spark, and Flink. Java APIs were chosen for development due to their availability and the author's familiarity.

The study's scope is limited to processing unstructured data using free tools in a single or pseudo-distributed cluster environment. Complex statistical analyses are avoided, focusing instead on comparing processing times.

IV. DEVELOPMENT PROPOSAL

This section describes details of the experiment involving the Hadoop, Spark and Flink platforms. The experiment will be a word counter, running on all platforms, and which will batch process unstructured data of variable sizes, in order to obtain their respective execution times.

A. General Configuration

The experiments and tools will be configured and emulated in a Linux environment using the Ubuntu version 16.04 distribution, with 8 GB of RAM and a 4-core processor. The 3 platforms will work in pseudo-distributed mode, in this mode a single node is used. Where HDFS will be used as a shared file system in conjunction with YARN, it will be possible to simulate an application running in a cluster.

B. Word Counter

The main reason for choosing this application to evaluate the platforms was the similarity of implementation using Hadoop, Spark and Flink, in addition to the fact that working with a word counting algorithm demands batch processing with large volumes of data, thus giving the opportunity to measure the result through the gain in execution time (speedup) of each platform under similar conditions.

As a data source, the Word Count program will use the federal constitution, made available in PDF format by the Federal Supreme Court (BRASIL, 1988), having 514 pages, and with a total size of 2.8MB. As it is a relatively small file, which does not allow the full potential of the study object tools to be extracted, it was decided that the file will be converted to text format (.txt), as in this format the file can be easily replicated multiple times, with the aim of arriving at an ideal size for the research objective and simulating large-scale

processing. To replicate the file, the following command was used in the Linux terminal:

```
for i in {1..N};do cat CONSTITUICAO.txt >>
FONTE1.txt;done
```

Where N is the number of times the text file will have its content concatenated³. To obtain a 1GB data source, the file was concatenated 1000 times, to obtain the second 3GB source, 3000 iterations were necessary, and finally the 5GB file with 4850 iterations.

V. DEVELOPMENT

This chapter will describe the development process of the experiment carried out using a word counting application. In addition to demonstrating some of the main configurations of the AH, AS and AF tools, to obtain a functional application where it will be possible to develop the proposed result.

A. Development History

As there are 3 tools discussed, each of which requires a certain type of configuration and requirements to be executed, some steps will not be detailed, as they would present very extensive content and will not influence the final result of the experiment.

B. Tools and Technologies

In addition to the tools that are part of the ecosystem of each platform, others were used that contributed to the experiment and which will be presented below.

C. Eclipse

Eclipse is an integrated development environment (IDE), considered one of the most popular IDEs for Java development. The software also provides an extensible system of plug-ins, which allows customization of the environment. Most of it is written in Java, but it is possible to use it to develop in other languages through plug-ins. Eclipse is free and open-source software, released under its own license terms. (ECLIPSE, 2017, our translation).

The Eclipse Neon version (4.6.3) was used, which already supports Java version 8. This software was used because it is a tool that greatly facilitates development in Java, it was also used because it has a plug-in for integration with Maven, this tool is used for compilation automation, and will be discussed in more detail below.

Apache Maven is a free build automation tool, primarily used in Java projects. We chose to work with Maven as it makes it much easier to compile and serve the application with the necessary dependencies to use the APIs of the platforms studied in this experiment.

Free online tool that allows you to convert PDF into text files, without the need to install software. It was necessary to use it to convert the experiment's data source and thus be able to manipulate it more easily. This tool is available at <<http://pdftotext.com>> (Accessed on October 15, 2017).

D. Development History

In this first part, the following initial requirements for installing the experiment execution environment will be described:

- System user inserted in the “Hadoop” group
- Java Development Kit (JDK)
- SSH client without password
- Environment variables mapped to Java, Hadoop and Yarn.

To run Hadoop and consequently HDFS, the application user must be in a group called “Hadoop”, therefore, instead of creating the group and adding the existing user to that group, it was chosen to create a new user called “hduser” and add it to the new “Hadoop” group.

Then, to run the experiment and consequently make all platforms work, it is necessary to install Java in the execution environment. The latest version of Java was chosen, version 8, which is compatible with all 3 platforms

Hadoop communicates through SSH (Secure Shell) authentication, so it must be installed and configured correctly. In the case of configuration, we chose to use a DSA public key for the hduser user to use the SSH service. This policy allows communication between nodes, without having to enter a username and password, as this type of communication between nodes is quite frequent, it was necessary to adopt this solution to avoid unnecessary overload due to authentication.

Next, it is necessary to configure the environment variables for the new user. To do this, you must access the `~/.bashrc` file and add the variables.

E. Hadoop

Having completed the first part of the installation, the steps followed to create the word counter application and run it in the Hadoop environment will now be described.

Hadoop has several configuration files in which various parameters are configured for the various components that run on Hadoop, such as HDFS, MapReduce, and YARN.

Next, it is recommended to use the command “`hdfs namenode -format`” to format the master node of our cluster, even if it has not been used, so if an error occurred in the Hadoop configuration, it can be identified in advance.

In a second run, it was also necessary to delete the datanode's temporary files, using the command “`rm -rf /usr/local/hadoop_data/hduser/datanode/*`”, without this step, the datanode node stopped working due to an error which occurred when trying to run the processing a second time, as some of the datanode's temporary data is not deleted at the end of a process.

Next, it was necessary to run the HDFS master and slave nodes, running the script: `$HADOOP_HOME/sbin/start-dfs.sh` and then the manager nodes provided by yarn, running the script: `$HADOOP_HOME/sbin/start-yarn.sh`. Figure 1 shows the result of the “`jps`” command, which lists all processes running on the JVM that are necessary to run an application in pseudo-distributed mode on the Hadoop platform.

```
hduser@vostro-5470:/usr/local/hadoop/etc/hadoop$ jps
6114 ResourceManager
5715 DataNode
6217 NodeManager
5609 NameNode
18954 Jps
5916 SecondaryNameNode
```

Fig. 1. Processes running on the JVM.

After starting all the processes necessary to run the Word Count application, the following commands in Figure 2 were executed to move the 3 files that will be used as a data source.

```
### 1 GB file
hadoop fs -copyFromLocal /home/hduser/inputs/wordcount/SOURCE1.txt
/user/hduser/wordcount/input

### 3GB file
hadoop fs -copyFromLocal /home/hduser/inputs/wordcount/SOURCE2.txt
/user/hduser/wordcount/input

### 5 GB file
hadoop fs -copyFromLocal /home/hduser/inputs/wordcount/SOURCE3.txt
/user/hduser/wordcount/input
```

Fig. 2. Experiment data source.

The implementation of Word Count, in Java, receives two parameters informing the path of the input file and the path of the output file. As HDFS was used as the default file system, the paths provided followed the pattern: `hdfs:///user/hduser/wordcount/*`. Next, the Maven tool was used to compile the Java program into a JAR file, which will be executed by Hadoop, using the command as shown in Figure 3.

```
hadoop jar /home/hduser/wordcount-hadoop-tcc.jar hadoop.WordCount
/user/hduser/wordcount/input/fonteN.txt /user/hduser/wordcount/hadoop/output
```

Fig. 3. Running the Wordcount Job in Hadoop.

F. Spark

To run the experiment on the AS platform, the file system initialization process was similar to that used for Hadoop, with only some specific configurations for the platform and for its integration with HDFS and its operation in pseudo-distributed mode. with the same computational resources used in the Hadoop configuration. To do this, it was necessary to change the variables in the file `$SPARK_HOME/conf/spark-env.sh`, as shown in Figure 4:

```
export SPARK_MASTER_IP=localhost
export SPARK_WORKER_CORES=1
export SPARK_WORKER_MEMORY=1000m
export SPARK_WORKER_INSTANCES=1
```

Fig. 4. Spark configuration in pseudo-distributed mode.

Subsequently, with the file system initialized and Hadoop configured, the Job was executed to count the words, using the command shown in the Figure 5:

```
spark-submit --class spark.WordCount --master yarn --deploy-mode cluster
/home/hduser/wordcount-spark-tcc.jar /user/hduser/wordcount/input/fonteN.txt
/user/hduser/wordcount/spark/output
```

Fig. 5. Running the Wordcount Job in Spark.

The result obtained in word frequencies was exactly the same as that obtained with the experiment carried out by MapReduce on Hadoop, but with different completion times between the platforms. For the ordered visualization, the command presented in Figure 6 was used.

```
hadoop fs -cat /user/hduser/wordcount/spark/output/* | sort -t, -n -k 2 -r | head -50
```

Fig. 6. Command to visualize the word count for Spark.

G. Flink

On the AF platform there were some difficulties in its configuration. First, there was incompatibility with version 2.8.X of Hadoop, with the latest version of Flink 1.3.2 (November 2017), so it was necessary to reinstall Hadoop to a previous compatible version, in this case version 2.7 was used. 4. After redoing the Hadoop configuration process, Flink was configured, where it was necessary to change the file `$FLINK_HOME/conf/flink-conf.yaml`, and add the variable `fs.hdfs.hadoopconf` pointing to the Hadoop configuration directory. The other settings were maintained following their standard parameterization.

To start the integration with YARN, it was necessary to run the script `$FLINK_HOME/yarn-session.sh -n 1` (where `-n` is the number of nodes we want to run in our YARN session).

It is worth reporting that there were some errors in keeping the Yarn session running, with some unexpected errors occurring that stopped the cluster from running.

The command in Figure 7 was then used to execute the word counter program.

```
flink run -m yarn-cluster -yn 1 -c flink.WordCount /home/hduser/wordcount-flink-tcc.jar --
input hdfs:///user/hduser/wordcount/input/fonteN.txt --output
hdfs:///user/hduser/wordcount/output
```

Fig. 7. Running the Wordcount Job in Flink.

As expected, at the end of the execution, the same word count frequency result was generated, as was the case on the other two platforms.

H. Results

The tests were repeated with at least 10 executions for each data source on each platform, so that an average processing time was obtained for each data source. However, there was an exception to the last data source on the Flink platform, due to the delay in completing the processing, so only 3 samples were collected for this specific case. This time was obtained through the web interface, where the HDFS file system runs.

Figures 8, 9, 10 present a base comparison between AH, AS and AF for the Word Count application with an input file of 1GB, 3GB and 5 GB respectively.

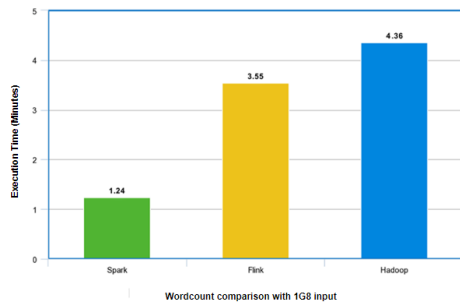


Fig. 8. Comparative execution time of the Word Count application with 1 GB of input

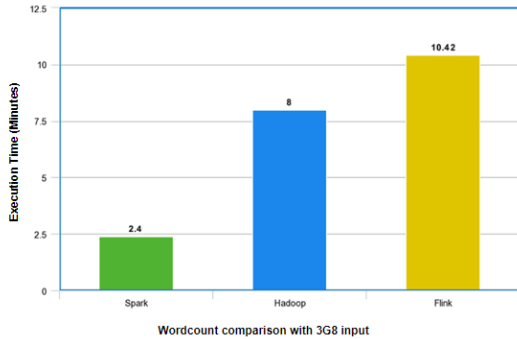


Fig. 9. Comparison of execution time of the Word Count application with 3 GB of input

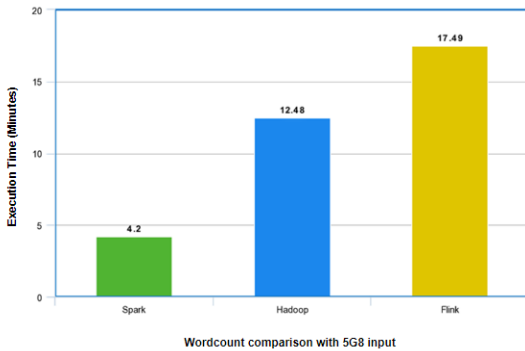


Fig. 10. Comparative execution time of the Word Count application with 5 GB of input

VI. CONCLUSION

As a result of this work, it was possible to get a sense of the demand generated by so-called Big Data and understand how a large-scale data processing architecture works, both in batch and in real time, even if the latter has not been explored in practice.

With the experiment it was also possible to analyze the execution time of the AH, AS and AF tools. In the first experiment with a 1 GB data source, it was already possible to notice a notable difference between Spark in relation to the other 2 competitors, in this first experiment Spark was on average 186% faster than Flink, and 251% faster than the pioneer Hadoop. In the second with a 3 GB data source, Hadoop was faster than Flink, but was considerably behind Spark, which proved to be 233% faster compared to Hadoop, and 334% compared to Flink. When processing the last 5 GB data source, it was possible to notice Spark's performance

improvement of 197% compared to Hadoop, for 316% of Flink's execution time.

Thus, it was possible to unanimously conclude that AS obtained a better result in batch processing this large amount of data in a cluster in pseudo-distributed mode.

This experiment was run in a minimal environment that did not provide a distributive property of execution, and there was no parallelism. This factor could directly influence the final result.

REFERENCES

- [1] N. Hasliza, M. Hassan, K. Ahmad, and H. Salehuddin, "Diagnosing the issues and challenges in data integration implementation in public sector," *International Journal on Advanced Science, Engineering and Information Technology*, vol. 10, no. 2, pp. 529–535, 2020.
- [2] A. Mohamed, M. K. Najafabadi, Y. B. Wah, E. A. K. Zaman, and R. Maskat, "The state of the art and taxonomy of big data analytics: view from new big data framework," *Artificial Intelligence Review*, vol. 53, no. 2, pp. 989–1037, 2020.
- [3] U. Islam, A. Muhammad, R. Mansoor et al., "Detection of distributed denial of service (DDoS) attacks in IOT based monitoring system of banking sector using machine learning models," *Sustainability*, vol. 14, no. 14, p. 8374, 2022.
- [4] A. A. Najar and S. Manohar Naik, "DDoS attack detection using MLP and random forest algorithms," *International Journal of Information Technology*, vol. 14, no. 5, pp. 2317–2327, 2022.
- [5] S. Ananthu, N. Sethumadhavan, and H. Narayanan Ag, "Credit card fraud detection using Apache Spark analysis," in 2021 5th international conference on trends in electronics and informatics (ICOEI), pp. 998–1002, Tirunelveli, India, 2021.
- [6] J. Wang, Y. Yang, T. Wang, R. S. Sherratt, and J. Zhang, "Big data service architecture: a survey," *Journal of Internet Technology*, vol. 21, no. 2, pp. 393–405, 2020.
- [7] M. S. Mahmud, J. Z. Huang, S. Salloom, T. Z. Emara, and K. Sadatdiynov, "A survey of data partitioning and sampling methods to support big data analysis," *Big Data Mining and Analytics*, vol. 3, no. 2, pp. 85–101, 2020.
- [8] N. J. Shoumy, L. M. Ang, K. P. Seng, D. M. Rahaman, and T. Zia, "Multimodal big data affective analytics: a comprehensive survey using text, audio, visual and physiological signals," *Journal of Network and Computer Applications*, vol. 149, article 102447, 2020.
- [9] D. Kshirsagar and S. Kumar, "An efficient feature reduction method for the detection of DoS attack," *ICT Express*, vol. 7, no. 3, pp. 371–375, 2021.
- [10] R. Sahal, J. G. Breslin, and M. I. Ali, "Big data and stream processing platforms for industry 4.0 requirements mapping for a predictive maintenance use case," *Journal of Manufacturing Systems*, vol. 54, pp. 138–151, 2020.
- [11] O. M. Araz, T. M. Choi, D. L. Olson, and F. S. Salman, "Role of analytics for operational risk management in the era of big data," *Decision Sciences*, vol. 51, no. 6, pp. 1320–1346, 2020.
- [12] R. H. Hamilton and W. A. Sodeman, "The questions we ask: opportunities and challenges for using big data analytics to strategically manage human capital resources," *Business Horizons*, vol. 63, no. 1, pp. 85–95, 2020.
- [13] M. Bansal, A. Goyal, and A. Choudhary, "A comparative analysis of K-nearest neighbour, genetic, support vector machine, decision tree, and long short term memory algorithms in machine learning," *Decision Analytics Journal*, vol. 3, article 100071, 2022.
- [14] L. Abualigah and B. Al Masri, "Advances in MapReduce big data processing: platform, tools, and algorithms," *Artificial Intelligence and IoT*, vol. 85, pp. 105–128, 2021.
- [15] N. AlNuaimi, M. M. Masud, M. A. Serhani, and N. Zaki, "Streaming feature selection algorithms for big data: a survey," *Applied Computing and Informatics*, vol. 18, no. 1/2, pp. 113–135, 2022.